Autonomous Parking

In this lesson, let’s learn how to detect parking signs using instructions.

1. Getting Ready

1) Before starting, ensure the map is laid out flat and free of wrinkles, with smooth roads and no obstacles. For specific map laying instructions, please refer to "Map Laying and Prop Installation" in the same directory as this section for guidance.

2) The roadmap model discussed in this section is trained using YOLOv5. For further information on YOLOv5 and related content, please refer to "16. Machine Learning”.

3) When experiencing this game, ensure it is conducted in a well-lit environment, but avoid direct light shining on the camera to prevent misrecognition.

2. Program Logic

Begin by capturing the real-time image from the camera and applying operations such as erosion and dilation.

Next, invoke the YOLOv5 model to compare the obtained image with the target screen image.

Finally, based on the comparison results, identify the parking sign and autonomously guide the robot to park in the designated parking space.

The source code of this program is saved in **/home/ubuntu/ros2_ws/src/example/example/yolov5_detect/yolov5_trt.py**

3. Operation Steps

---

1. Note: The following steps exclusively activate road sign detection in the return screen, without executing associated actions. Users seeking direct experience with

autonomous driving may bypass this lesson and proceed to "Integrated Application" within the same file.

2. When inputting commands, please ensure accurate differentiation between uppercase and lowercase letters as well as spaces. Utilize the "Tab" key for keyword completion.

---

1) Start the robot, and access the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

3) Execute the command to disable the app auto-start service.

~/.stop_ros.sh

```
~/.stop_ros.sh
```

4) Run the command to open the camera node.

ros2 launch peripherals depth_camera.launch.py

```
ros2 launch peripherals depth_camera.launch.py
```

5) Open a new command line terminal. Enter the command to enable the camera node.

ros2 launch yolov5_ros2 yolov5_ros2.launch.py model:="traffic_signs_640s_7_1"

```
ros2 launch yolov5_ros2 yolov5_ros2.launch.py model:="traffic_signs_640s_7_1"
```

6) Open a new command line terminal. Execute the command to open the interface for live camera feed.

rqt

```
rqt
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '
tmp/runtime-ubuntu'
```

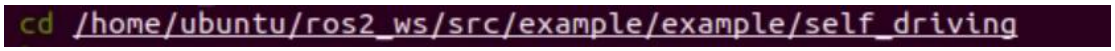
7) Position the road sign in front of the camera, and the robot will recognize the road sign and mark it on the image . If you need to terminate this game, press “Ctrl+C”. If it fails, please try again.

Note: In the event that the model struggles to recognize traffic-related signs, it may be necessary to lower the confidence level. Conversely, if the model

consistently misidentifies traffic-related signs, raising the confidence level might be advisable.

8) Run the command to navigate to the directory containing programs.

```
cd /home/ubuntu/ros2_ws/src/example/example/self_driving
```



9) Execute the command to open the game program.

```
vim self_driving.launch.py
```

The red box represents the confidence value, which can be adjusted to modify the effectiveness of target detection.

Once you've completed the game experience, you can initiate the app service by following the instructions or restarting the robot.

If the app service is not activated, the associated app functions will be disabled. (The app service will automatically start when the robot restarts).

To restart the app service, enter the following command and wait for the buzzer to emit a single beep, indicating that the service startup is complete.

```
sudo systemctl restart start_node.service
```

4. Program Outcome

After initiating the game, position the robot on the road within the map. As the robot progresses towards the parking sign, it will automatically park in the designated parking space based on the instructions provided by the road sign.

5. Parameter Adjustment

If the robot stops upon recognizing the parking sign and the parking position is not optimal, adjustments to the parameters in the program source code can be made.

1) Click-on  to open the command-line terminal.

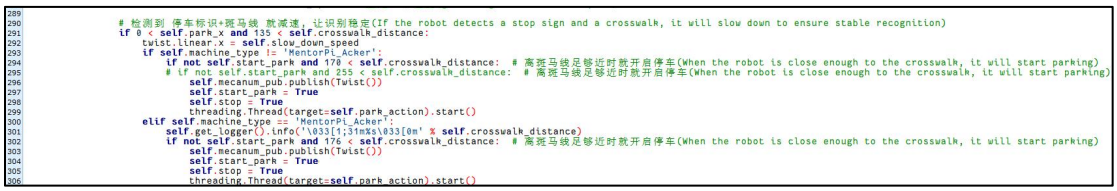
2) Execute the command to navigate to the directory containing game programs.

```
cd ros2_ws/src/example/example/self_driving/
```

3) Run the command to access the source code.

```
vim self_driving.py
```

4) Press the "i" key to enter insert mode and locate the code within the red box. Adjusting the parameters within the red box allows you to control the starting position for the robot to initiate the parking operation. Decreasing the parameters will result in the robot stopping closer to the zebra crossing, while increasing them will cause the robot to stop further away. Once adjustments are made, press the "ESC" key, type ":wq", and press Enter to save and exit.



```
289
290             # 检测到 停车标识+斑马线 就减速，让识别稳定(If the robot detects a stop sign and a crosswalk, it will slow down to ensure stable recognition)
291             if 0 < self.park_x and 135 < self.crosswalk_distance:
292                 twist.linear.x = self.slow_down_speed
293                 if self.machine_type != 'MentorPi_Acker':
294                     if not self.start_park and 170 < self.crosswalk_distance:  # 离斑马线足够近时就开启停车(When the robot is close enough to the crosswalk, it will start parking)
295                     # if not self.start_park and 255 < self.crosswalk_distance:  # 离斑马线足够近时就开启停车(When the robot is close enough to the crosswalk, it will start parking)
296                         self.mecanum_pub.publish(Twist())
297                         self.start_park = True
298                         self.stop = True
299                         threading.Thread(target=self.park_action).start()
300                 elif self.machine_type == 'MentorPi_Acker':
301                     self.get_logger().info('\033[1;31m%s\033[0m' % self.crosswalk_distance)
302                     if not self.start_park and 176 < self.crosswalk_distance:  # 离斑马线足够近时就开启停车(When the robot is close enough to the crosswalk, it will start parking)
303                         self.mecanum_pub.publish(Twist())
304                         self.start_park = True
305                         self.stop = True
306                         threading.Thread(target=self.park_action).start()
```

You can adjust the parking processing function to alter the parking position of the robot. Initially, the parking action sets the linear speed in the negative direction of the Y-axis (right of the robot) to 0.2 meters per second, with a forward movement time of (0.38/2) seconds. To position the robot in the ideal location on the left side of the parking space, modify the speed and time accordingly.

```
197     # 泊车处理(parking processing)
198     def park_action(self):
199         if self.machine_type == 'MentorPi_Mecanum':
200             twist = Twist()
201             twist.linear.y = -0.2
202             self.mecanum_pub.publish(twist)
203             time.sleep(0.38/0.2)
204         elif self.machine_type == 'MentorPi_Acker':
205             twist = Twist()
206             twist.linear.x = 0.15
207             twist.angular.z = twist.linear.x*math.tan(-0.5061)/0.145
208             self.mecanum_pub.publish(twist)
209             time.sleep(3)
210 
211             twist = Twist()
212             twist.linear.x = 0.15
213             twist.angular.z = -twist.linear.x*math.tan(-0.5061)/0.145
214             self.mecanum_pub.publish(twist)
215             time.sleep(2)
```